NKT
Tag 2: Methoden, Objekte, Klassen und Arrays

Alexa Katharina-Ronja Brockmöller, Tobias Wrede

5. März 2026

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Überschreiben von Methoden
   - String Klasse
   - Standarteingabe
2. Stack
   - Heap und Stack
   - Stack
   - Heap

Gliederung

1. Objekte und Klassen
   - Klassen
   - Überladen von Methoden
   - Überschreiben von Methoden
   - String Klasse
   - Standarteingabe
2. Stack
   - Heap und Stack
   - Stack
   - Heap

Klassen

- Eine Klasse kann erstellt werden
- Die erstellte Klasse wird Objekt genannt
- Welche Methoden auf dem Objekt aufgerufen werden können, steht in der Klasse. Klasse $\rightarrow$ Bauplan für Objekt
- Es kann nur EINE Klasse mit dem Namen X geben, jedoch unendlich viele Objekte der Klasse X
- Alle Klassen erben von der Klasse Object (Vererbung kommt später)

Methoden in Klassen

- Wollen wir ein Objekt einer Klasse erstellen benutzen wir das Schlüsselwort "new". (Deswegen bitte keine variablen "new"nennen!)
- Falls wir keinen Konstruktor selbst schreiben, existiert Default Konstuktor
- Da jede Klasse von Object erbt, hat auch jedes Objekt eine toString() Methode
- Methodennamen Werden im UpperCamelCase benannt

Wie müsste die Person Klasse aussehen, damit folgendes funktioniert?

```
Person p1 = new Person();
Person p2 = new Person("Superman");
System.out.println(p1.toString() + "," + p2.toString());
//Ausgabe soll "Kein Name,Superman" sein
```

Klassen

```
class Person {
    String name = "Kein Name";
    Person(){}
    Person(String name){
        this.name = name;
    }
    @Override
    public String toString(){
        return name;
    }
}
```

Methoden Überladen

In Java können wir Methoden überladen. Dies haben wir bereits in der Person Klasse getan. Der Konstruktor existiert zwei mal. Einmal als Default Konstruktor und einmal als Konstruktor der einen Namen für die Person entgegen nimmt und diesen Speichert. Überladen ist sehr hilfreich:

```
int sum(int a, int b) {
    return a + b;
}

double sum(double a, double b) {
    return a + b;
}
```

Nun können wir die Methode sum mit int wie auch mit double verwenden.

Überschreiben von Methoden

Heinrich Heine Universität Düsseldorf

Das Überschreiben von Methoden wirft im Gegensatz zum Überladen die alte Methode weg. Es existieren also nur Methoden, die nicht überschrieben wurden. In unserem Person Beispiel haben wir die toString() Methode, die wir von Object geerbt haben, überschrieben. Wenn wir nun auf den Person Objekten die toString() Methode aufrufen wird nun unsere neue Methode genutzt, da die Alte überschrieben ist.

```
@Override
public String toString(){
    return name;
}
```

[1] Was wäre Ausgegeben worden, wenn wir die toString() Methode nicht überschrieben hätten?

[1] @Override Annotation nicht nötig, dennoch sinnvoll

Methoden der String Klasse

Heinrich Heine Universität Düsseldorf

Ein Beispiel für eine bereits bestehende Klasse ist die String Klasse.

```
String msg = "funnymsg,          irgendwas";
String msgBig = msg.toUpperCase();
String[] msgSplit = msg.split(",");
String msgTrim = msgSplit[1].trim();
```

Da ein String ein Objekt ist, also kein Primitiver Datentyp können wir Strings nicht mit == vergleichen. Damit vergleichen wir nur ob die Referez die selbe ist, oder nicht. Wie vergleichen wir nun also Strings?

```
String a = "Hallo";
String b = "Welt";
System.out.println(a.equals(b));
```

Standardeingabe

Wir können auch die Standardeingabe benutzen, um unser Programm mit Daten aufzurufen. Dafür benötigen wir Scanner:

```
import java.util.Scanner;
class Wuerfel {
    static void main(String[] args) {
        Scanner stdin = new Scanner(System.in);
        while(stdin.hasNext()){
            int number = Integer.parseInt(stdin.nextLine());
            while(number > 0){
                for(int i = 0; i < number; i++){
                    System.out.print("*");
                }
                number--;
                System.out.println();
```

Scanner & Dateien

Wir können Scanner aber auch auf spezifische Dateien setzen. Dafür brauchen wir jedoch den Dateipfad.

```
Scanner scanner = new Scanner(dateipfad);
```

Achtung! Wenn an diesem Pfad keine Datei liegt, wird unser Programm abstürzen. Wir sollten deshalb einen *try-catch* Block benutzen:

```
try{
    Scanner scanner = new Scanner("/NKT/wichtigeDatei.txt");
} catch(Exception e){
    e.printStacktrace();
}
```

Gliederung

1 Objekte und Klassen
- Klassen
- Überladen von Methoden
- Überschreiben von Methoden
- String Klasse
- Standarteingabe

2 Stack
- Heap und Stack
- Stack
- Heap

Heap und Stack

Wo werden eigentlich die Daten gespeichert?

- Stack
- Heap

Stack

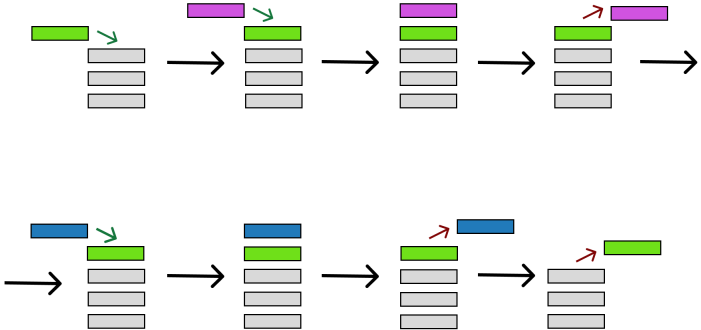
Heinrich Heine Universität Düsseldorf

Auf dem Stack Speichern wir primitive Datentypen und Referenzen. Auch Rücksprungadressen etc. (siehe Rechenarchitektur).

- float
- int
- double
- boolean
- char

Der Stack ist schneller erreichbar als der Heap. Dafür aber deutlich kleiner.

Stack - Zugriff



LIFO - Last In first Out

Heap

Heinrich Heine Universität Düsseldorf

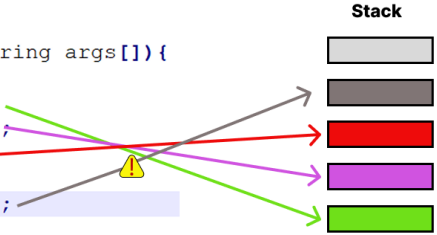
Auf dem Heap Speichern wir Objekte.

- Arrays
- String
- Integer
- Double
- Person
- eigene Klassen

Der Heap ist im Gegensatz zum Stack viel größer. Dafür aber langsamer.

Heap

```
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}
```



Vom Objekt wird nur eine Referenz gespeichert!

Referenzen

```
Person a = new Person();
    Person b = new Person();
    System.out.println(a == b);
```

Wird true oder false ausgegeben?

Referenzen

```
Person a = new Person();
Person b = new Person();
System.out.println(a == b);
```

Wird true oder false ausgegeben?
→ Es wird false ausgegeben

```
System.out.println(a);          // Person@7ad041f3
System.out.println(b);          // Person@251a69d7
```

Heap 01

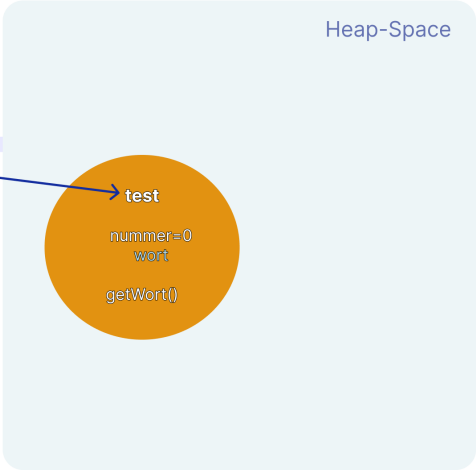
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();



    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```

Heap-Space

Heap 02

- Durch Aufruf des Konstruktors wird ein neues Objekt auf dem Heap erzeugt

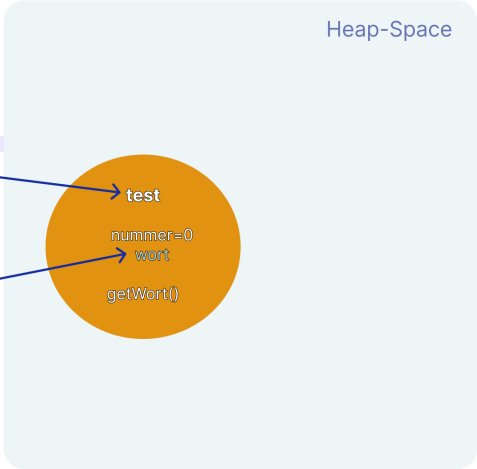
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



Heap 03

- Gilt auch für Objekte in einem Objekt
- Jede Zuweisung gibt nur die Referenz auf das Objekt weiter!

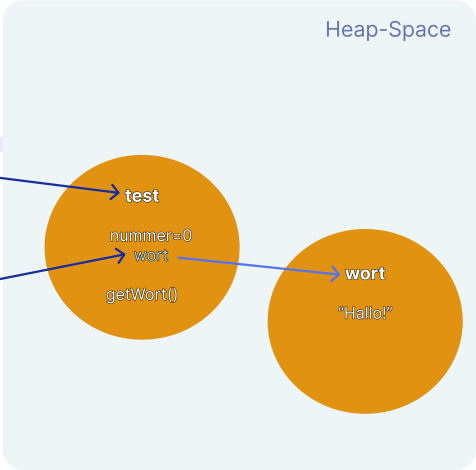
```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}

public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```



Heap 04

- Shallow Copy:
  Wenn das Objekt kopiert wird, werden nur Referenzen auf andere Objekte kopiert

- Deep Copy:
  Die Objekte im Objekt sind echte Kopien

```java
public class Main{

    public static void main(String args[]){

        int zahl = 2;
        double kommaZahl = 0.5;
        char buchstabe = 'a';

        Test test = new Test();

    }
}
```

```java
public class Test{
    public int nummer=5;
    private String wort = "Hallo!";

    public String getWort(){
        return this.wort;
    }
}
```